第 7 章推導解答

DS07-01 • 穩定性與逆序數

三輪分別為 `(2,B),(5,A),(4,C),(2,D)`；`(2,B),(4,C),(5,A),(2,D)`；`(2,B),(2,D),(4,C),(5,A)`。逆序對為 A-B、A-C、A-D、C-D，共 4，右移次數為 1+1+2=4。B-D 鍵相同不計嚴格逆序。若改用 ≥，D 插入時會越過 B，破壞等鍵原順序，也不再能把搬移次數等同嚴格逆序數。

DS07-02 • 三向分割與堆疊

初始 lt=i=0、gt=6。讀 4 前進 i；讀 2 交換到索引 0，lt=1、i=2；讀 4 前進 i=3。讀 7 時 gt=5，交換索引 3、5 成 `[2,4,4,4,1,7]`，i 不變。讀到 4 前進，讀到 1 與 lt=1 交換，得到 `[2,1,4,4,4,7]`、lt=2、i=gt=5，完成。每次真正遞迴子區至多為目前非等鍵區的一半，沿堆疊大小以幾何速率下降，故深度 O(log n)。若每次分割極偏，較大側雖改成迴圈，仍需處理 n、n−1、⋯ 次掃描，總 $\Theta(n^2)$。

DS07-03 • 比較下界的模型

對互異 n 鍵，每種相對排列都需不同葉節點，共 n! 葉。二元比較樹高度至少 $\log_2(n!)$，由 n!≥(n/2)^(n/2) 得 Ω(n log n)。計數排序用鍵值直接索引，已超出只允許比較獲取資訊的模型。所有鍵相等時，排序輸出不用區分 n! 種鍵序，資訊量不同，所以不能拿互異鍵下界宣稱此單一輸入也必花 n log n。下界仍是一般問題的最壞界。

DS07-04 • LSD 穩定性的反例

個位穩定排序為 `31,11,22,14`。十位排序應讓十位 1 的兩鍵維持 11 在 14 前，得到 `11,14,22,31`。若同十位反轉，得到 `14,11,22,31`，並未完整排序。對固定鍵寬 w，d 約為 $\text{ceil}(w/\log_2 b)$；增大 b 可減少輪數，卻增加每輪計數陣列初始化及空間，總 Θ(d(n+b))。最適 b 依 n、鍵寬及記憶體條件而定，不能只追求最少輪數。

第 7 章推導解答 （續）

原創驗算程式附錄: sorting.cpp

```
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <random>
#include <stdexcept>
#include <utility>
#include <vector>

void insertion_sort(std::vector<int>& values) {
	for (std::size_t i = 1; i < values.size(); ++i) {
		const int key = values[i];
		std::size_t position = i;
		while (position > 0 && values[position - 1] > key) {
			values[position] = values[position - 1];
			--position;
		}
		values[position] = key;
	}
}

void merge_range(std::vector<int>& values, std::vector<int>& buffer,
	std::size_t left, std::size_t right) {
	if (right - left < 2) {
		return;
	}
	const auto middle = left + (right - left) / 2;
	merge_range(values, buffer, left, middle);
	merge_range(values, buffer, middle, right);
	std::size_t a = left;
	std::size_t b = middle;
	for (std::size_t out = left; out < right; ++out) {
		if (a < middle && (b == right || values[a] <= values[b])) {
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 7 章推導解答 （續）

```
                        buffer[out] = values[a++];
                } else {
                        buffer[out] = values[b++];
                }
        }
        std::copy(buffer.begin() + static_cast<std::ptrdiff_t>(left),
                buffer.begin() + static_cast<std::ptrdiff_t>(right),
                values.begin() + static_cast<std::ptrdiff_t>(left));
}

void merge_sort(std::vector<int>& values) {
        std::vector<int> buffer(values.size());
        merge_range(values, buffer, 0, values.size());
}

void quick_range(std::vector<int>& values, std::size_t left, std::size_t right) {
        while (right - left > 1) {
                const int pivot = values[left + (right - left) / 2];
                std::size_t lower = left;
                std::size_t cursor = left;
                std::size_t greater = right;
                while (cursor < greater) {
                        if (values[cursor] < pivot) {
                                std::swap(values[lower++], values[cursor++]);
                        } else if (values[cursor] > pivot) {
                                std::swap(values[cursor], values[--greater]);
                        } else {
                                ++cursor;
                        }
                }
                if (lower - left < right - greater) {
                        quick_range(values, left, lower);
```

第 7 章推導解答 （續）

```
                        left = greater;
                } else {
                        quick_range(values, greater, right);
                        right = lower;
                }
        }
}

void quick_sort(std::vector<int>& values) {
        quick_range(values, 0, values.size());
}

struct Record {
        int key;
        int id;
};

void stable_insertion_sort(std::vector<Record>& records) {
        for (std::size_t i = 1; i < records.size(); ++i) {
                const Record item = records[i];
                std::size_t position = i;
                while (position > 0 && records[position - 1].key > item.key) {
                        records[position] = records[position - 1];
                        --position;
                }
                records[position] = item;
        }
}

void require(bool condition, const char* message) {
        if (!condition) {
                throw std::runtime_error(message);
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 7 章推導解答 （續）

```
        }
}

int main() {
        std::vector<std::vector<int>> cases{{}, {1}, {2, 2, 2}, {5, 2, 4, 2},
                {1, 2, 3, 4, 5}, {5, 4, 3, 2, 1}, {0, -5, 3, -5, 9}};
        std::mt19937 random(20261024);
        for (int trial = 0; trial < 180; ++trial) {
                std::vector<int> values(random() % 160);
                for (auto& value : values) {
                        value = static_cast<int>(random() % 41) - 20;
                }
                cases.push_back(values);
        }
        for (const auto& input : cases) {
                auto expected = input;
                std::sort(expected.begin(), expected.end());
                for (auto sorter : {insertion_sort, merge_sort, quick_sort}) {
                        auto actual = input;
                        sorter(actual);
                        require(actual == expected, "sort differs from reference permutation");
                }
        }
        std::vector<Record> records{{5, 0}, {2, 1}, {4, 2}, {2, 3}};
        stable_insertion_sort(records);
        require(records[0].id == 1 && records[1].id == 3 && records[2].id == 2 &&
                records[3].id == 0, "equal-key stability was lost");
        std::cout << "PASS sorting: " << cases.size()
                << " cases for each of 3 algorithms; record stability\n";
}
```

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study